Branch History Table Prediction of Moving Target Branches Due to Subroutine Returns

David R. Kaeli †‡
Philip G. Emma ‡

Rutgers University †
Dept. of Electrical and Computer Engineering
New Brunswick, N.J. 08903

IBM ‡
T. J. Watson Research Center
P.O. Box 218
Yorktown Heights, NY 10598

Abstract

Ideally, a pipeline processor can run at a rate that is limited by its slowest stage. Branches in the instruction stream disrupt the pipeline, and reduce processor performance to well below ideal. Since workloads contain a high percentage of taken branches, techniques are needed to reduce or eliminate this degradation.

A Branch History Table (BHT) stores past action and target for branches, and predicts that future behavior will repeat. Although past action is a good indicator of future action, the subroutine CALL/RETURN paradigm makes correct prediction of the branch target difficult. We propose a new stack mechanism for reducing this type of misprediction.

Using traces of the SPEC benchmark suite running on an RS/6000, we provide an analysis of the performance enhancements possible using a BHT. We show that the proposed mechanism can reduce the number of branch wrong guesses by 18.2% on average.

Introduction

Ideal speedup in pipeline processors is seldom achieved due to stalls and breaks in the execution stream. These interruptions are caused by data dependencies, hardware resource contention, and control transfers (branches and interrupts). Of these pipeline hazards, the control transfer class can be the most detrimental to pipeline performance. We address this class in this paper.

A taken branch in the instruction stream will introduce a break in execution. In order to approach peak performance, the pipeline must constantly be kept full of valid instructions. The frequency of taken branches is shown to be over 20% of the instructions in today's workloads [1][2][3][4]. It is necessary to reduce the number of pipeline stalls due to taken branches if we wish to approach ideal performance.

A Branch History Table (BHT) as been show to be an effective mechanism for reducing the number of these stalls. In this paper we characterize the different wrong predictions made by a BHT. We discuss why a design should focus not on the amount of saved history, but instead on the correctness of the saved branch target address. We further provide a solution to eliminating one class of incorrect target address predictions by using a simple subroutine CALL/RETURN detection scheme.

1. Branch Penalty Reduction Techniques

Since branches can degrade the performance of the pipeline severely, many methods have been proposed to address this problem. Techniques such as delayed branching [5], branch folding [6], static prediction based on opcode, and branch target prefetching [7], have all been been used to reduce the penalty imposed by branches.

Delayed branches have been implemented in many architectures (i.e. IBM 801, MIPS, Intel 80860, etc.). The idea with a delayed branch is to schedule useful work during the time in which the branch instruction is being resolved. One problem with this technique is that for a large percentage of the branches, an instruction can not be found to fill the delay slot. (Campbell reports that 10% of the unconditional branch slots and 40-60% of the conditional branch slots can not be filled[8].)

Branch folding, as implemented in the CRISP microprocessor, attempts to include the next instruction

© 1991 ACM 0-89791-394-9/91/0005/0034 $1.50

address within the instruction format. A field is added to each instruction that contains the address of the next instruction to be executed. As the instructions are decoded, the next instruction address is immediately available. On an unconditional branch, the target address is entered into the next address field of the previous instruction. The branch is effectively folded into the previous instruction. On a conditional branch, the next address field of the previous instruction is modified once the upstream condition that the branch depends upon is resolved. In this case some delay will be introduced unless the condition code can be resolved far enough upstream to permit the modification of the next address field before the branch is executed. Reordering the instruction sequence to do this is called branch spreading. Due to the clustering of branches, this technique can only work in a small number of cases.

Some static prediction techniques can produce a high percentage of correct guesses, but their success is highly dependent on the nature of the workload. Smith has reported that by statically predicting that every branch is taken, on average 76.7% of the predictions were correct [9]. Smith has also shown that opcode-based branch prediction can yield accuracies in the range of 65.7% to 99.4%. The problem with this approach is that it relies heavily on characteristics of the branches in the workload; these characteristics tend to vary from workload to workload.

Prefetching branch targets, once the a branch has been decoded and the target address generated, was implemented on the IBM System 360 Model 91. For forward branches (backward branches are handled by loop-mode) the branch sequencer initiates two doubleword fetches down the target stream. The target instruction can then be rerouted to the decoder once the branch is resolved. One problem with this strategy is that unnecessary memory accesses for instructions that are not executed can drastically reduce the amount of memory bandwidth available to the processor.

A common problem with all of these techniques is that prediction or redirection is performed at decode or execute time. By this time, a typical processor pipeline has prefetched and possibly decoded past this control point. Cache and memory cycles will be used for fetching instructions that will not be executed (assuming prefetch is simply fetching the next sequential instruction.)

A Branch History Table operates ahead of decode-time via the autonomous prediction and prefetching of historically observed branch target instructions. Appendix A provides an example of one BHT design. The BHT redirects the prefetching of the instruction stream as early as possible. Providing the ability to predict far upstream of the execution unit will minimize any latency incurred by the target fetch. None of the other mechanisms we described issue the target fetch as early as in a BHT design.

In the next section we provide further insight into the use of a BHT.

2. Branch History Tables

One approach to reducing the penalty incurred by taken branches is to examine the problem from a programming level. By studying some of the more common programming constructs in high-level languages, we propose a branch penalty reduction strategy that can take advantage of typical branching behavior. Unconditional branches, such as subroutine CALL/RETURN and GOTO, are always taken. Their behavior seems quite easy to predict. Conditional branching constructs, such as the DO LOOP and the IF/THEN/ELSE block, do not always follow the same path. A DO LOOP is a fairly well-behaved structure, usually resulting in a taken branch to a destination at a negative displacement off the program counter. The IF/THEN/ELSE block may not exhibit a predictable behavior, since it may change direction on subsequent iterations.

Unfortunately, a processor does not have information about the high-level language programming constructs that are used to produce the object-runable code. The processor is only able to detect patterns of branch behavior by capturing a sample of the behavior. Then, by using this stored history of prior branch execution, the processor can predict subsequent execution direction when the branch is next encountered. This is the fundamental concept behind a Branch History Table.

Branch History Tables have been described by Smith[9], Lee and Smith[10], Holgate and Ibbett[11], Hughes etal.[12], Lilja[13], and Pomerene[14]. In it's simplest form, a BHT maintains the outcomes of previously executed branches. The table is accessed by the instruction prefetch unit and decides whether prefetching should be redirected or not. The table is searched for a valid entry, just as a cache is searched. The table is typically set-associative, as is the case with with many cache organizations [15]. An entry is only added to the table when a taken branch is executed by the processor.

On each BHT hit, the historical information in that entry is used by the prediction algorithm. The algorithm redirects prefetching for a taken prediction, or continues with the next sequential instruction for a not-taken prediction. Some implementations invalidate the entry when the branch changes to not taken. In this case, a BHT miss will occur subsequently, and next-sequential prefetching will ensue. If the prediction is wrong, the processor must be equipped with a back-out strategy to restore the necessary state.

The amount of branch history kept also affects the accuracy of the predictions made. Lee and Smith[10] describe how to use five bits of history to increase the prediction accuracy over 92% for the majority of their workload (the five bits indicate the direction of the last five iterations of a branch). It can be argued that one or at most two bits of history are necessary to take advantage of the underlying data structures. Two bits would be useful to handle the case of nested loops. Any more history than

two bits may even degrade the prediction performance for other workloads.

Some design trade-offs need to be addressed to tailor the BHT to a particular machine implementation. Most of these trade-offs can be determined by analyzing traces of the target machine environment. Issues such as table size, associativity, number of history bits saved, and prediction algorithm all influence the success of a particular implementation. The goal is to provide the processor with the correct instruction stream before the "control point"[11] in the pipeline is reached. Other factors that determine the effectiveness of a BHT are: the aging out of valid branch history table entries, the frequency of first-time taken branches, instruction address aliasing, and incorrect predictions due to page relocation. It should be stressed that all of these implementation issues are tied directly to the nature of the target workload.

Previous BHT studies focused on the accuracy of the prediction algorithm [9][10]. Prediction algorithm accuracy only relates to predicting conditional branches. Unconditional branches should always be predicted to be taken. One key problem that has been overlooked in the past is the correctness of the branch target address that is stored in the BHT. The BHT logic must be able to predict the direction of the branch (taken or not taken), and must also produce the correct target address. Then, instruction prefetching can be redirected to the appropriate target address.

Wrong predictions due to changing history typically accounted for more than half of the wrong predictions in our traces. We analyzed what caused the remainder of the wrong predictions and found that a large percentage of them could be attributed to a changing branch target. (In two of our traces, over 50% of the incorrect predictions were due to changing branch target.) From further analysis, we found that incorrect predictions made due to CALL/RETURN from subroutines were the major contributor to this problem. An important difference between changing history and a changing target is that we can eliminate many of the wrong guesses due to changing target using a set of CALL/RETURN stacks. This design is detailed in Section 4.

3. Trace Tapes and Model Description

To analyze the performance of a BHT, we choose to use full instruction traces. If only address traces were used, then only taken branches could be detected, and misses in the BHT on not-taken conditional branches could not be detected. One feature of a BHT is that by invalidating not-taken conditional branches in the table, they are (by default) predicted not taken.

The traces used in this evaluation are taken from workload running on an IBM RS/6000[4] running AIX. The applications traced are taken from the SPEC version 1.0 benchmark suite[16]. Appendix B provides the details of each trace. The traces are taken from both Fortran and C source programs. Ten traces, 1 million instructions each, were used to run through our BHT model.

Our BHT model is designed with the following parameters: 1) the table is fully associative and each entry contains the full branch address as the tag along with the full branch target address, 2) since we only use history of the last taken branch in our prediction algorithm, no branch history bits are necessary in our implementation, 3) if a branch is in the table then it must have been taken last time, and 4) replacement is LRU. As discussed before, only in some cases can two bits be used effectively, and any more than two are highly sensitive to workload characteristics.

We assume in our model that: 1) that no overlays (due to dynamic memory remapping) occur, 2) the BHT only keeps information for taken branches, 3) if a valid entry is found in the BHT and if the current execution of the branch results in a not taken branch, then the BHT entry is invalidated, and 4) when the branch target address changes, the corresponding target field is updated in the BHT.

First we look at how the size of the BHT affects prediction accuracy. Our prediction results are broken down into the following five classes: 1) branches predicted correctly, 2) branches predicted incorrectly due to a change in target address, 3) branches predicted incorrectly due to incorrect history, 4) branches that miss the BHT that are not taken (miss correctly), and 5) branches that miss the BHT that are taken (miss incorrectly). A miss in a BHT should not be confused with a miss in a cache. A miss in a BHT is the desired result for branches that are not taken.

BHT sizes of 128, 256, 512, 1024, 2048 and 4096 entries were simulated. These sizes were chosen as reasonable sizes to implement in today's technology. It can be argued that a fully-associative design is overkill (in practice a 2 or 4 way design would be more practical). Our purpose is not to propose a optimal BHT associativity scheme. Instead we show how the BHT performs and we identify the action taken for each branch encountered on the trace. This breakdown is shown in the next section for each of the ten traces.

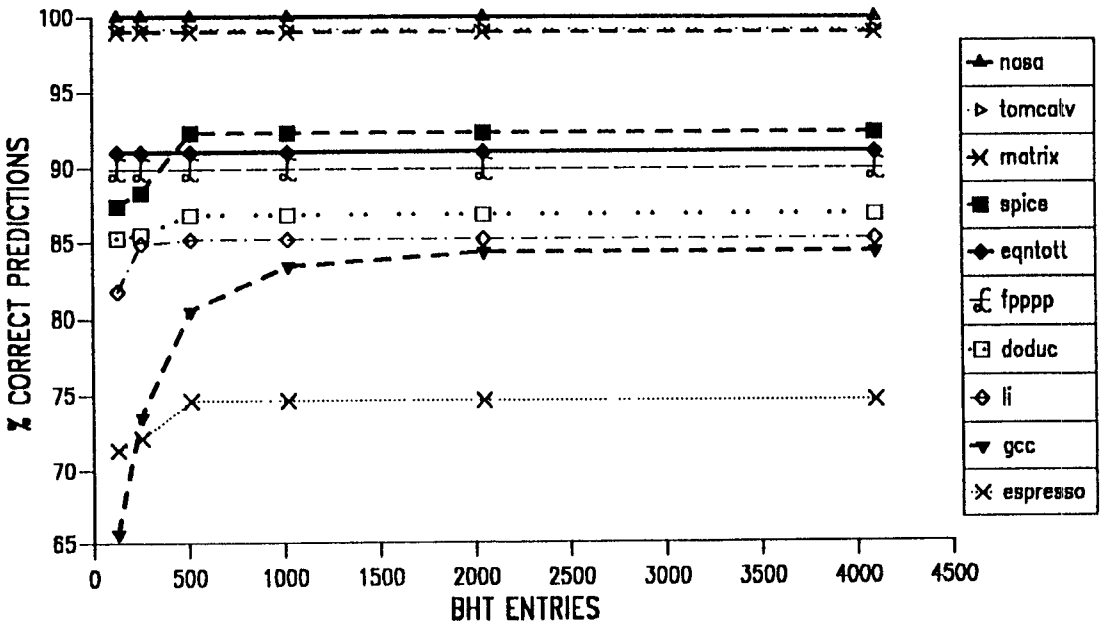

Figure 1. Percent Correct Predictions (includes all branches)

4. BHT Modeling Results

Figure 1 plots the correct predictions as a percent of all branches encountered on each tape. This number includes BHT misses (which result in not-taken predictions) for not-taken branches. These are implicitly correct.

The BHT accuracy is better than 85% for most of the traces. The worst accuracy is 70-75% for the Espresso benchmark. The poor performance on Espresso can be attributed to the large number of misses, which are a result of BHT invalidations due to wrong history predictions (a number of branches are displaying an unpredictable behavior.) GCC also exhibits poor performance for BHT's smaller than 1024 entries. This can be attributed to the large number of unique branches found in this benchmark.

The effect of increasing the size of the BHT past 512 entries does not increase the prediction accuracy significantly. From Table 1 we see that no trace contains more than 4096 unique branches. Thus, increasing the BHT will not change the prediction accuracy. Our purpose here is not to propose the optimal BHT size. Results will vary from workload to workload. Table 1 is used to here to pick a BHT size that captures the maximum number of branches contained in any single trace of our trace suite. (This emulates an infinite BHT.)

APPLICATION	BRANCHES
tomcatv	2
nasa	7
matrix	10
eqntott	135
fpppp	160
spice	350
espresso	485
doduc	648
li	1,038
gcc	2,529

Table 1. Number of Unique Branch Instructions

In Figure 2 we show branch performance for each of the ten traces when run through a 4096-entry BHT. We select this breakdown so that we can identify where there exists potential for improvement. The goal is to decrease the number of wrong predictions. As previously stated, we can not expect to perform better across all workloads if we only focus on the history algorithm..

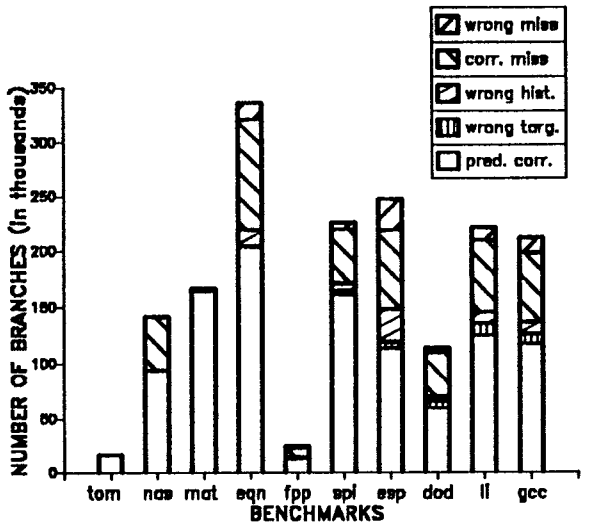

Figure 2. BHT Performance Profile (4096 BHT entries)

The number of incorrect misses encountered can not be reduced. We have already captured the maximum number of branches possible by using a sufficiently large BHT. Wrong misses are a result of the first time misses (either due to the first occurrence of the branch on the trace tape or a branch being taken after it had been invalidated in the BHT due to wrong history).

A point that should be stressed is when the BHT predicts that a branch will be taken and then it is not taken, the BHT will perform worse than the non-BHT case. (The non-BHT design will have prefetched the next sequential instruction address.) For a wrong miss, the BHT will perform the same as the non-BHT case. (The non-BHT design will have prefetched past the branch, which is the same action taken by the BHT design.)

As a result, we focus on decreasing the number of incorrect predictions. In the next section, we look at this problem and give a solution that reduces the number of wrong predictions due to incorrect target address.

5. CALL/RETURN Stack Pair Implementation

For each of the ten traces we have plotted the breakdown of all wrong predictions in Figure 3. This illustrates the potential for improvement. In six out of ten of the traces there exists potential for reducing the number of wrong branch-target predictions. The other four traces contain very few wrong predictions; we must be sure not to reduce the existing accuracy with any algorithm used to optimize the other six traces.

To reduce the number of wrong predictions due to changing targets, we need to better understand the underlying structure that causes these targets to change. Subroutine RETURN constitutes the largest percentage of wrong targets.

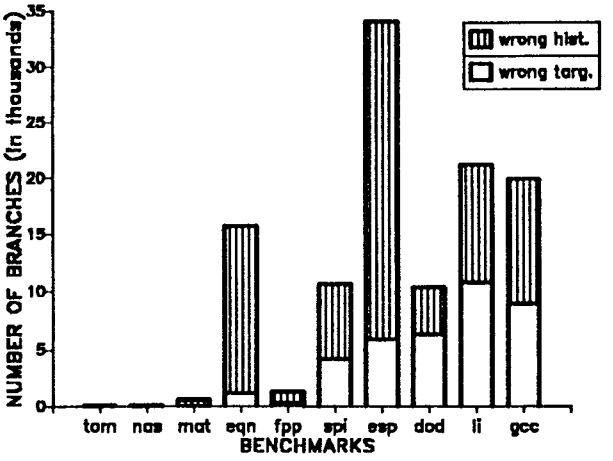

Figure 3. BHT Wrong Prediction Profile (4096 BHT entries)

The SAMPLE program provided in Figure 4 illustrates the problem. From address 100 we branch to the subroutine PRINT at address 500. We then enter an entry in the BHT for address 100 with a target 500. On the RETURN from PRINT we enter the address 600 in the BHT with a target 110. On instruction 130 we again branch to PRINT, entering into the BHT the address 130 with a target 500. When we get to the end of the subroutine PRINT, we will hit on a valid entry in the BHT (address = 600) and predict that we will return to address 110. Of course, this is not correct. The next instruction to be executed is at address 140.

```
PROGRAM SAMPLE

Instruction
Address

 100                CALL @PRINT
 110                  .
 120                  .
 130                CALL @PRINT
 140                  .
   .                  .
   .                  .
   .                  .
   .

PRINT:
 500                  .
 510                  .
   .                  .
   .                  .
   .                  .
 600                RETURN
```

Figure 4. Sample Program

How can we detect this behavior and supply the BHT with the correct address before an incorrect prediction is made? By using a set of stacks, we can identify when a CALL is made and then supply the correct branch target when the RETURN is encountered [17].

Figure 5 shows a picture of the two-stack design. In addition to the two stacks, each BHT entry must be augmented with a "subroutine return" bit (SR) that is used to indicate that this is a special entry.

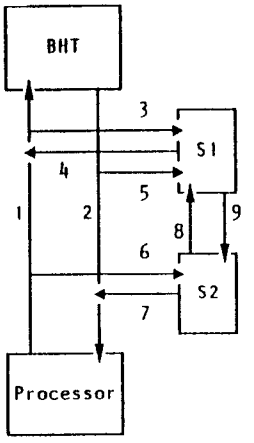

Figure 5. Call/Return Stack Design

We will use the programming example in Figure 4 to describe the design. The algorithm works as follows:

1. When the CALL at address 100 is executed, the addresses 100 and 500 are sent along path 1 to create an entry with the branch address set to 100, and the branch target set to 500. Also, the target address of the CALL (500) is pushed onto the S1 stack via path 3 and the return address (110, the address that is sequential to the CALL instruction) is pushed onto the S2 path via path 6.

2. When the RETURN is executed at 600 with target address 110, the addresses 600 and 110 are sent to the BHT along path 1, branch address and branch target respectively. In parallel, 110 is sent to stack S2 along path 6 to see if S2 has an entry for it. In this case, S2 does have an entry and its corresponding entry in S1 (in this case 500) is found via path 8 and sent along path 4 where it replaces the target address on path 1. Then the entry in the BHT has a branch address of 600 and a target address of 500. A bit in the BHT (the SR bit) is turned on to denote that this is a special entry.

3. When the CALL at address 130 is executed, the address (130) and target (500) are sent to the BHT via path 1. Also, the target address of the CALL (500), is pushed onto stack S1 on path 3 and the return point (140) is pushed on stack S2 on path 6.

4. When instruction address 600 is later prefetched, the BHT will find an entry for address 600 (the RETURN), and that entry will have the SR bit turned on. The prediction associated with the entry (branch address set to 600 and branch target set to 500) is sent to path 2, and since the special bit is on, the target field is sent to stack S1 on path 5 to see if S1 has an entry for 500. In this case, S1 does and so the corresponding entry in S2 (140) is identified via path 9 and is put on path 7 where it replaces the target field on path 2. (All entries on the stack are compared in parallel. In the event of a tie, the most recent entry is chosen.)

5. The prediction that is made for address 600 will then have the target 130. The stack provides the correct address instead of the historical address.

To show how much savings can be expected from this implementation, we have run all ten traces through our 4096 entry BHT model, adding an SR bit for each entry and a pair of CALL/RETURN stacks. Stack depths of 5 and 10 were simulated. Table 2 lists the results. By adding a 5-entry pair of stacks we can reduce the percentage of wrong predictions up to 4.6% (an improvement of 31.5%). We have found that for many of the benchmarks, if we use the stack pair with a BHT containing 128 entries, it will predict more branches correctly than using a stackless BHT with 4096 entries (i.e. LI run with a 128-entry BHT and a pair of 5-entry stacks showed a 33% improvement over a stackless 4096-entry BHT).

	NO STACK	5 ENTRIES	10 ENTRIES
tomcatv	0.4	0.0	0.0
nasa	0.0	0.0	0.0
matrix	0.0	0.0	0.0
eqntott	7.1	7.0	7.0
fpppp	9.3	8.0	8.0
spice	6.2	4.9	4.9
espresso	23.1	20.2	20.2
doduc	14.8	13.4	13.4
li	14.6	10.0	8.9
gcc	14.5	12.5	11.2

Table 2. Percent of Wrong Predictions (includes all predictions)

Doubling the stack depth to 10 entries only showed a difference in the LI and GCC benchmarks. Another encouraging result is that performance of the four benchmarks that were previously generating good results were not adversely affected by the introduction of the stacks.

Conclusions

To optimize processor pipeline performance, the instruction pipeline must be kept full with valid instructions. Much effort has been spent investigating different approaches to reducing or eliminating latencies that arise in the presence of taken branches. Branch History Tables have been shown to be an effective approach to reducing these latencies. BHT's are especially attractive since they attempt to redirect prefetching as early as possible.

Previous BHT studies have focused on the problem of trying to optimize the prediction algorithm by saving different amounts of history. In our study, we have characterized the BHT behavior while concentrating on those aspects of the design that reduce the number of incorrect predictions, and that are less dependent on workload characteristics than some previously proposed history-based algorithms. We have also presented a robust BHT design that is based on the underlying programming structures that exist in today's applications. We have shown how to reduce the number of wrong predictions made by 18.2% on average, by using a pair of CALL/RETURN stacks. The results have been found to consistently perform as well if not considerably better across all of the traces in our study.

Acknowledgements

The authors would like to thank Juho Tang for providing the tracing tool used to produce the traces and to Herbert Freeman for his support of this research.

References

1. Kaeli D.R., Kirkpatrick S., Ong S., "PC Workload Characterization", Proceedings of the ACM Sigmetrics and Performance '89", May 89, pp.220.

2. Adams T., Zimmerman R., "An Analysis of 8086 instruction usage in MS DOS programs", Proc. Third Symposium on Architectural Support For Programming Languages and Operating Systems, Boston, 1989, pps.152-161.

3. Clark D.W., Levy H., "Measurement and Analysis of Instruction Use in the VAX 11/780", Proc. Ninth Symposium on Computer Architecture, Austin, Tx., April 1982, pps.9-17.

4. Grohoski G.F., "Machine Organization of the IBM RISC System/6000 processor", IBM Journal of Research and Development, Vol.34, No.1, Jan. 1990, pps.37-58.

5. Patterson D.A, "Reduced Instruction Set Computers", Communications of the ACM, Vol.28-1, Jan. 1085, pps.8-21.

6. Ditzel D.R., McLellan H.R., "Branch Folding in the Crisp Microprocessor: Reducing Branch Delay to Zero," Proc. 14th Ann. Symp. Computer Arch., 1987, pps.2-9.

7. Anderson D.W. etal., "The IBM System/360 Model 91: Machine philosophy and instruction handling", IBM Journal of Research and Development, Jan. 1967, pps.8-24.

8. Campbell R., "Compiling C for the Reduced Instruction Set Computer", Master's report, EECS, U.C. Berkeley 94720, Dec. 1980.

9. Smith J.E., "A Study of Branch Prediction Strategies", Proc. Eighth Symposium on Computer Architecture, May 1981, Minneapolis, pps.135-148.

10. Lee J., Smith A.J., "Branch Prediction Strategies and Branch Target Buffer Design", Computer 17:1, Jan. 1984, pps.6-22.

11. Holgate R.W., Ibbett R.N., "An Analysis of Instruction-Fetching Strategies in Pipelined Computers", IEEE Trans. on Computers, Vol.C-29, No.4, April 1980, pps. 325-329.

12. Hughes J.F., Liptay J.S., Rymarczyk J.W., Stone S.E., "Multi-Instruction Stream Branch Processing Mechanism", U.S. Patent 4,200,927, Apr. 29, 1980.

13. Lilja D.J., "Reducing the Branch Penalty in Pipelined Processors", IEEE Computer Magazine, July 1988, pps.47-55.

14. Pomerene J.H., Puzak T.R., Rechtschaffen R.N., Rosenfeld P.L., Sparacio F.J., "Pageable Branch History Table", U.S. Patent 4,679,141, Jul. 7, 1987.

15. Smith A.J., "Cache Memories", Computing Surveys, Vol.14 No.3, Sept. 1982. pps.473-530.

16. SPEC Quarterly Newsletter, System Performance Evaluation Cooperative, 1st Quarter 1990.

17. Webb C.F., "Subroutine Call/Return Stack", IBM Tech. Disc. Bullen., Vol.30, No.11, April 1988.

Appendix A

Branch History Table Description

The purpose of the Branch History Table is to redirect prefetching such that when a branch is executed in the processor, the target instruction stream has already been fetched and is ready for execution. The example below is one possible implementation of a BHT.

In our example, we assume that prefetching will fetch the next sequential address in the absence of any redirection. In the non-BHT description, prefetching is redirected only when the execute unit (E) detects a taken branch.

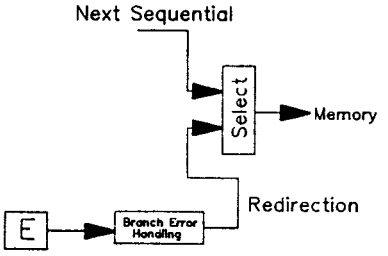

Figure A.1. Non-BHT Description

In the BHT description, each prefetch effects a BHT lookup. The IPFAR (Instruction Prefetch Address Register) is compared against the branch address (BA). If a match is found, this constitutes a BHT hit, and the corresponding target address (TA) is feed to the select logic. Prefetching is redirected to the new address.

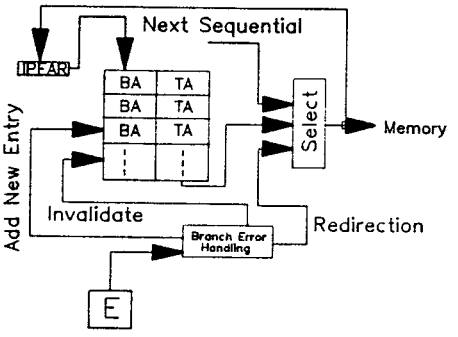

Figure A.2. BHT Description

In the case that the BHT prediction is wrong, the execute unit (E) will signal the branch error handling logic to redirect prefetching to the new target address or to the fall-through address (just as in the case of the non-BHT description). If the instruction was neither detected in the BHT nor signaled as wrong by the execute unit, prefetching continues with the next sequential address. When a first-time taken branch is encountered, the BHT is updated by the branch error handling logic. Similarly, when a wrong prediction is made by the BHT, the branch error handling logic will invalidate the corresponding BHT entry.

Appendix B

Trace Tape Description

The benchmarks used in this study were taken from the version 1 release of the SPEC Benchmark Suite. The traces are taken at an offset of 1 million instructions into each application. Each trace is 1 million instructions in length.

1. Espresso - One of a collection of tools for the generation and optimization of Programmable Logic Arrays. This is a integer benchmark written in C. Our trace used tial.in as input.

2. Spice - A general purpose circuit simulation program written in Fortran.

3. Doduc - A Monte Carlo simulation involving scalar floating-point execution, written in Fortran.

4. Nasa - A collections of 7 floating point kernels, comprised of 2,200 lines of Fortran.

5. Li - A Lisp interpreter written in C, solving the 8-queens problem.

6. Eqntott - An integer-intensive benchmark written in C, translating a logical representation of a boolean equation to a truth table.

7. Matrix - A vectorizable Fortran scientific benchmark.

8. Fpppp - A double precision Fortran quantum chemistry benchmark.

9. Tomcatv - A double precision vectorized mesh generation Fortran program.

10. GCC - This is the GNU C compiler, compiling 19 preprocessed source files into optimized assembly language. There are 109,000 lines of C.